

BioInsight 360

A Privacy-First, Browser-Native Bioinformatics Platform for RNA-Seq Differential Gene Expression Analysis

**A Dissertation Submitted in Partial Fulfillment of the
Requirements for the Degree of Master of Science**

Submitted by: [Your Name]

Supervisor: [Supervisor Name]

Department: [Department Name]

University: [University Name]

September 2026

Declaration

I hereby declare that this dissertation titled "BioInsight 360: A Privacy-First, Browser-Native Bioinformatics Platform for RNA-Seq Differential Gene Expression Analysis" is a record of the original research work carried out by me under the supervision of [Supervisor Name], and has not been submitted for the award of any other degree or diploma.

All sources of information and intellectual contributions have been duly acknowledged and cited.

Signature: _____

Date: September 2026

Abstract

The rapid evolution of RNA-Sequencing (RNA-Seq) technology has transformed the field of transcriptomics by enabling genome-wide quantification of gene expression at single-nucleotide resolution. However, current web-based bioinformatics platforms that facilitate RNA-Seq analysis rely on traditional client-server architectures. This dependency introduces two critical challenges: (1) data privacy risks arising from the transmission of sensitive clinical genomic data to external servers, and (2) performance bottlenecks caused by server queuing during peak usage.

This dissertation presents the design, implementation, and evaluation of **BioInsight 360**, a novel bioinformatics platform that fundamentally shifts the computational paradigm by migrating all analytical processing from the backend server to the client's web browser. Built using the Next.js framework and leveraging modern HTML5 Web APIs, the platform executes data parsing (via PapaParse), three distinct normalization methods (Counts Per Million, Upper Quartile, Median of Ratios), two statistical tests (Welch's t-test, Mann-Whitney U test), and Benjamini-Hochberg FDR correction entirely within the browser's JavaScript engine. Because all computation occurs locally, the raw count matrices never leave the user's machine, achieving zero-server data privacy.

Furthermore, BioInsight 360 introduces a **Multi-Method Consensus Engine** that simultaneously evaluates six distinct analytical pipelines (3 normalizations x 2 statistical tests) and aggregates their results. Inter-method agreement is quantified

using Fleiss' Kappa and pairwise Jaccard Similarity, allowing users to distinguish high-confidence differentially expressed genes (DEGs) from method-sensitive false positives. The platform's interactive dashboard visualizes results through Volcano plots, Agreement Heatmaps, and UpSet plots rendered with Plotly.js.

Keywords: RNA-Seq, Differential Gene Expression, Edge Computing, Privacy-First Bioinformatics, JavaScript, Next.js, Consensus Analysis

Acknowledgements

I would like to express my sincere gratitude to my supervisor, [Supervisor Name], for their guidance and support throughout this research. I am also thankful to the faculty of [Department Name] at [University Name] for providing an intellectually stimulating environment.

This project was built entirely on open-source technologies, and I would like to acknowledge the developers and maintainers of Next.js, React, PapaParse, and Plotly.js for making their tools freely available.

Finally, I thank my family and friends for their constant encouragement and patience during the completion of this dissertation.

Table of Contents

1. **Chapter 1: Introduction** — Background, Problem Statement, Objectives, Dissertation Outline
2. **Chapter 2: Literature Review** — Traditional RNA-Seq Pipelines, Existing Web Platforms, Edge Computing in Bioinformatics
3. **Chapter 3: System Architecture & Methodology** — Framework Selection, State Management, Statistical Engine, Consensus Algorithm
4. **Chapter 4: Implementation** — Codebase Walkthrough, Upload Module, Normalization, Statistical Testing, Consensus Engine
5. **Chapter 5: Software Testing & Validation** — Performance Benchmarking, Accuracy Validation, Edge-Case Testing
6. **Chapter 6: Results & Discussion** — Case Study, Biological Interpretation, Limitations
7. **Chapter 7: Conclusion & Future Scope**
8. **References**

Chapter 1: Introduction

1.1 Background and Motivation

The history of transcriptome profiling dates back to the development of microarray technologies in the 1990s. For over a decade, microarrays were the dominant technology for measuring gene expression levels across the genome. However, microarrays are limited by their reliance on predefined probes, their susceptibility to cross-hybridization artifacts, and their narrow dynamic range (Mortazavi et al., 2008).

The introduction of Next-Generation Sequencing (NGS) platforms in the mid-2000s, particularly Illumina's sequencing-by-synthesis technology, gave rise to RNA-Sequencing (RNA-Seq). Unlike microarrays, RNA-Seq does not require prior knowledge of the genome sequence, offers single-nucleotide resolution, provides a digital measurement of transcript abundance (as opposed to analog fluorescence intensities), and has a vastly superior dynamic range spanning five orders of magnitude (Wang et al., 2009).

RNA-Seq has since become the gold standard for transcriptomics. A typical RNA-Seq experiment involves:

1. Extracting total RNA from biological samples (e.g., tumor tissue vs. healthy tissue).
2. Reverse-transcribing the RNA into cDNA and fragmenting it.

3. Sequencing the cDNA fragments on an NGS platform to generate millions of short reads (FASTQ files).
4. Aligning these reads to a reference genome using tools such as STAR (Dobin et al., 2013) or HISAT2 (Kim et al., 2019).
5. Quantifying the number of reads mapping to each gene using featureCounts (Liao et al., 2014) or Salmon (Patro et al., 2017) to produce a **count matrix**.
6. Performing **Differential Gene Expression (DGE) analysis** on the count matrix to identify genes whose expression levels differ significantly between experimental conditions.

DGE analysis is the most critical downstream step, as it directly informs biological discovery — identifying potential biomarkers, therapeutic targets, or elucidating the molecular mechanisms underlying disease.

1.2 The Bioinformatics Bottleneck

While the cost of generating RNA-Seq data has plummeted — the cost of sequencing a human genome has dropped from approximately \$100 million in 2001 to under \$1,000 in 2022 (National Human Genome Research Institute) — the cost and complexity of **analyzing** that data has not decreased proportionally.

Traditionally, DGE analysis is performed using specialized R/Bioconductor packages:

- **DESeq2** (Love et al., 2014): Models count data with a negative binomial distribution and uses shrinkage estimation for dispersion parameters.
- **edgeR** (Robinson et al., 2010): Uses empirical Bayes methods and exact tests for negative binomial data.
- **limma-voom** (Ritchie et al., 2015): Transforms RNA-Seq count data into continuous, log-normal values using precision weights, enabling the use of linear modeling.

While these tools are mathematically rigorous and widely cited (DESeq2 alone has over 40,000 citations as of 2024), they all require proficiency in R programming. This creates a significant barrier for bench biologists, clinical researchers, and students who lack computational training but are the primary generators and consumers of RNA-Seq data.

1.3 The Problem with Existing Web Platforms

To bridge this gap, several web-based bioinformatics platforms have been developed:

- **Galaxy** (Afgan et al., 2018): A comprehensive open-source platform that wraps command-line bioinformatics tools in a web-based GUI. While powerful, Galaxy requires users to upload their data to public or institutional servers. For large RNA-Seq datasets (often several gigabytes), the upload process itself can take hours. Furthermore, computational jobs are queued on shared servers, leading to significant wait times during peak usage.

- **iDEP** (Ge et al., 2018): An R/Shiny-based web application that integrates many exploratory data analysis (EDA) and DGE tools. iDEP is user-friendly but suffers from session timeouts and server crashes when processing large datasets, as the computation is entirely server-side.
- **NetworkAnalyst** (Xia et al., 2015): Focuses on network-based meta-analysis of gene expression data. Like iDEP, it transmits all data to a central server.

All three platforms share a fundamental architectural limitation: they rely on a **client-server model** where the user's data is uploaded to and processed on a remote server. This architecture introduces two critical challenges:

1.3.1 Data Privacy and Regulatory Compliance

Clinical RNA-Seq data is inherently sensitive. In the context of precision medicine, RNA-Seq is increasingly performed on patient-derived tumor biopsies, where the expression profiles can be linked to individual patients. Uploading such data to third-party cloud servers — even for analysis purposes — raises significant concerns under:

- **HIPAA** (Health Insurance Portability and Accountability Act) in the United States, which mandates strict controls over the handling and transmission of Protected Health Information (PHI).
- **GDPR** (General Data Protection Regulation) in the European Union, which requires explicit consent for

the processing of personal data and imposes restrictions on cross-border data transfers.

Many academic researchers, particularly those in clinical settings, are therefore hesitant or institutionally prohibited from using web-based tools that require data upload.

1.3.2 Server Bottlenecks and Scalability

Bioinformatics algorithms — particularly normalization and statistical testing across tens of thousands of genes — are computationally expensive. When multiple users submit jobs simultaneously on a public server (such as Galaxy's usegalaxy.org), jobs are placed in a queue and may take hours to complete. This makes iterative, exploratory analysis — a cornerstone of good bioinformatics practice — impractical.

1.4 Problem Statement

There is an urgent need for an accessible, GUI-driven bioinformatics platform that empowers non-programmers to perform sophisticated DGE analysis **without compromising data privacy**. The core research question addressed by this dissertation is:

How can we democratize complex RNA-Seq differential gene expression analysis through a web interface while guaranteeing absolute, zero-server data privacy?

1.5 Proposed Solution: BioInsight 360

To address these challenges, this project presents **BioInsight 360**, a privacy-first, browser-native bioinformatics platform. The key innovation is the complete migration of the computational pipeline from the backend server to the client's web browser using modern JavaScript and HTML5 Web APIs — an approach rooted in the emerging paradigm of **Edge Computing**.

In BioInsight 360, when a user uploads a count matrix CSV file, the file is read directly into the browser's memory using the HTML5 File API. The data is parsed using PapaParse (Holt, n.d.), normalized, and subjected to statistical testing — all within the browser's V8 JavaScript engine. At no point does the data leave the user's machine or traverse the network. The results are rendered interactively using Plotly.js.

Furthermore, to address the well-known problem of method-specific false positives in DGE analysis (i.e., genes that appear significant under one statistical test but not another), BioInsight 360 implements a **Multi-Method Consensus Engine**. This engine runs six distinct analytical pipelines (3 normalization methods x 2 statistical tests) in parallel and aggregates the results. Genes are classified by their **consensus score** — the number of pipelines in which they achieve statistical significance. Inter-method agreement is quantified using **Fleiss' Kappa** (a standard measure of inter-rater reliability) and **pairwise Jaccard Similarity**.

1.6 Aims and Objectives

The primary aim of this dissertation is to design, develop, and validate BioInsight 360 as a secure, high-performance, and user-

friendly alternative to traditional backend-reliant bioinformatics tools. The specific objectives are:

1. **Develop a Serverless Architecture:** To engineer a web application using Next.js (App Router) that operates entirely on the client-side, utilizing an in-memory Map and `sessionStorage` for state management, ensuring no data is transmitted to external servers.
2. **Implement In-Browser Statistical Pipelines:** To translate standard bioinformatics algorithms — CPM, Upper Quartile, and Median of Ratios normalization; Welch's t-test; Mann-Whitney U test; Benjamini-Hochberg FDR correction — into optimized, pure JavaScript.
3. **Design a Multi-Method Consensus Algorithm:** To create a logical engine that evaluates differential expression across six pipelines and calculates inter-method agreement using Fleiss' Kappa and Jaccard Similarity.
4. **Build an Intuitive User Interface:** To design a highly interactive dashboard utilizing Plotly.js for Volcano plots, Heatmaps, and UpSet plots.
5. **Conduct Performance and Accuracy Validation:** To benchmark the browser's memory consumption and execution speed, and to validate the mathematical accuracy of the in-browser statistical tests.

1.7 Dissertation Outline

The remainder of this dissertation is structured as follows:

- **Chapter 2 (Literature Review)** critically examines the mathematical foundations of DESeq2, edgeR, and limma-voom; compares existing web platforms; and explores the emerging role of Edge Computing in bioinformatics.
- **Chapter 3 (System Architecture & Methodology)** details the technical design of BioInsight 360, including the Next.js framework, state management strategy, and the mathematics behind each in-browser statistical implementation.
- **Chapter 4 (Implementation)** provides a complete codebase walkthrough, presenting the actual source code for the normalization, statistical testing, and consensus engine modules.
- **Chapter 5 (Software Testing & Validation)** presents the results of performance benchmarking and discusses the validation strategy.
- **Chapter 6 (Results & Discussion)** analyzes a case study and discusses the biological validity of the tool and its current limitations.
- **Chapter 7 (Conclusion & Future Scope)** summarizes the contributions and proposes future enhancements.

Chapter 2: Literature Review

2.1 RNA-Seq: From Raw Reads to Count

Matrices

The RNA-Seq workflow produces a **count matrix** — a two-dimensional table where rows represent genes (typically 15,000–25,000 for the human genome) and columns represent biological samples. Each cell contains a non-negative integer representing the number of sequencing reads that aligned to that gene in that sample. This count matrix is the starting point for all downstream DGE analysis.

A critical characteristic of RNA-Seq count data is that it is **discrete** (integer counts), **overdispersed** (the variance exceeds the mean, violating the Poisson assumption), and affected by **library size** (the total number of reads sequenced per sample, which is a technical artifact, not a biological signal).

2.2 Mathematical Foundations of Standard

DGE Tools

2.2.1 DESeq2

DESeq2 (Love et al., 2014) is arguably the most widely used tool for DGE analysis. Its statistical model consists of three key components:

Negative Binomial Model: DESeq2 assumes that the count K_{ij} for gene i in sample j follows a negative binomial (NB) distribution:

$$K_{ij} \sim \text{NB}(\mu_{ij}, \alpha_i)$$

where μ_{ij} is the expected count (fitted mean) and α_i is the gene-specific **dispersion parameter**. The variance of the NB distribution is:

$$\text{Var}(K_{ij}) = \mu_{ij} + \alpha_i \cdot \mu_{ij}^2$$

This formulation elegantly captures the overdispersion observed in RNA-Seq data: when α_i is small, the variance approaches the Poisson limit (μ); when α_i is large, the variance grows quadratically.

Median of Ratios Normalization: DESeq2 normalizes for library size by computing size factors using the Median of Ratios method. For each gene i , a pseudo-reference sample is constructed as the geometric mean across all samples. The size factor s_j for sample j is the median of the ratios of its counts to the pseudo-reference:

$$s_j = \text{median}_i \left(\frac{K_{ij}}{\left(\prod_{j=1}^m K_{ij} \right)^{1/m}} \right)$$

Shrinkage Estimation: For genes with very low counts, the maximum likelihood estimate of α_i can be unreliable. DESeq2 addresses this by fitting a trend of dispersion vs. mean across all genes, and then shrinking individual estimates toward this trend using an empirical Bayes procedure. This stabilizes the estimates for lowly expressed genes.

2.2.2 edgeR

edgeR (Robinson et al., 2010) also models counts using the negative binomial distribution but differs in its approach to dispersion estimation. edgeR estimates a common dispersion parameter across all genes and then uses the Cox-Reid profile-adjusted likelihood to estimate tagwise (gene-specific) dispersions. The tagwise dispersions are then moderated (shrunk) toward the common dispersion using an empirical Bayes approach.

For testing, edgeR uses exact tests analogous to Fisher's exact test, but adapted for the negative binomial distribution. This provides exact p-values for the two-group comparison case.

2.2.3 limma-voom

The limma package (Ritchie et al., 2015) was originally developed for microarray data, which is continuous and approximately normally distributed. To apply limma to RNA-Seq count data, the **VOOM** transformation is used:

1. Counts are normalized (typically using TMM — Trimmed Mean of M-values) and transformed to log-CPM values.
2. A mean-variance trend is estimated from the data.
3. Each observation is assigned a **precision weight** based on this trend.
4. These weights are incorporated into a standard linear model, allowing the use of the moderated t-statistics from the eBayes procedure.

The voom approach is attractive because it reduces the DGE problem to a well-studied linear modeling framework with decades of statistical theory behind it.

2.3 Comparison of Existing Web-Based Bioinformatics Platforms

Feature	Galaxy	iDEP	NetworkAnalyst
Architecture	Client-Server	Client-Server (R/Shiny)	Client-Server
Data Upload Required?	Yes	Yes	Yes
Privacy	Server-dependent	Server-dependent	Server-dependent
Statistical Engine	R (DESeq2, edgeR)	R (multiple)	R
Multi-Method Consensus	No	No	No
Queuing / Wait Times	Yes (can be hours)	Yes (session timeouts)	Yes
HIPAA/GDPR Compliant	Depends on deployment	No (public server)	No

Table 2.1: Comparative analysis of existing web-based bioinformatics platforms.

2.4 Edge Computing in Bioinformatics

Edge computing refers to the paradigm of performing computation at or near the source of data generation, rather than transmitting data to a centralized cloud server. In the context of web applications, the "edge" is the user's web browser.

Historically, browsers were considered too resource-constrained for scientific computing. However, several developments have changed this landscape:

1. **V8 JavaScript Engine:** Google's V8 engine, which powers Chrome and Node.js, employs Just-In-Time (JIT) compilation to translate JavaScript into highly optimized machine code at runtime. Modern V8 can achieve execution speeds within 2–5x of native C++ for numerical workloads.
2. **WebAssembly (Wasm):** Introduced in 2017, WebAssembly provides a low-level binary format that can execute at near-native speed in the browser. Tools like Emscripten can compile C/C++ code to Wasm, potentially allowing existing bioinformatics tools to run client-side in the future.
3. **Web Workers:** The Web Workers API enables multi-threaded execution in the browser by spawning background threads that do not block the main UI thread. This is critical for maintaining a responsive user interface during heavy computation.
4. **HTML5 File and Streams APIs:** The File API allows web applications to read local files without uploading

them to a server. The Streams API enables efficient, chunk-based processing of large files.

BioInsight 360 leverages all of these capabilities (except Wasm, which is reserved for future work) to create a fully functional bioinformatics pipeline that runs entirely in the browser.

Chapter 3: System Architecture & Methodology

3.1 Technology Stack

BioInsight 360 is built using the following technology stack:

Component	Technology	Justification
Framework	Next.js 16 (App Router)	React-based, supports both SSR and CSR; ideal for SPAs
UI Library	React 19	Virtual DOM for reactive state management
Data Parsing	PapaParse 5.5	Fastest in-browser CSV parser; RFC 4180 compliant
Visualization	Plotly.js 3.7 / react-plotly.js	Scientific-grade interactive plots
Decompression	HTML5 DecompressionStream API	Native .gz file decompression without libraries
State Management	Custom lib/storage.js	In-memory Map + sessionStorage fallback

Proxy (GEO only)	Next.js API Route (/api/ncbi)	CORS bypass for public NCBI data only
HTTP Client	Axios 1.19	Used only for the GEO metadata fetch proxy

Table 3.1: Technology stack used in BioInsight 360.

3.2 High-Level Architecture

BioInsight 360 follows a strictly **client-side rendering (CSR)** model for all analytical modules. The application consists of five primary modules:

1. **Upload Module** (/upload): Handles local file upload, GEO public dataset import, and pre-computed DEG integration.
2. **Quality Control Module** (/qc): Computes library size distributions, detection rates, sample correlations, and PCA.
3. **Consensus Module** (/consensus): Runs all six pipelines and displays the consensus dashboard.
4. **Pathways Module** (/pathways): Performs Gene Ontology enrichment analysis via the QuickGO API.
5. **Export Module** (/export): Generates downloadable PDF reports.

3.2.1 State Management: Zero-Server Data Persistence

A critical design decision was the elimination of any server-side database. Instead, BioInsight 360 uses a custom storage abstraction layer (`lib/storage.js`) that combines:

1. **An in-memory JavaScript Map** : Provides unlimited storage capacity (bounded only by the browser's available RAM) and instant read/write access. This is the primary data store and persists across Next.js client-side navigations.
2. **sessionStorage** as a persistence fallback: The storage module attempts to write data to `sessionStorage` (which survives page refreshes within a single browser tab) but silently catches quota errors. This ensures that data is preserved across F5 refreshes for smaller datasets, while gracefully degrading for larger ones.

This dual-layer approach solves two problems simultaneously: it avoids the 5–10 MB hard limit of `sessionStorage` alone (which would make large count matrices unloadable), and it ensures that data persists across page navigations without requiring a backend database or API calls.

3.2.2 NCBI GEO Proxy: The Only Server-Side Component

The only server-side code in BioInsight 360 is a lightweight Next.js API route (`/api/ncbi`) that acts as a **CORS proxy** for downloading publicly available datasets from the

NCBI Gene Expression Omnibus (GEO). This proxy is necessary because browsers enforce Cross-Origin Resource Sharing (CORS) policies that prevent client-side JavaScript from directly fetching data from `ftp.ncbi.nlm.nih.gov`.

Critically, this proxy:

- Does **not** store any data on the server.
- Acts purely as a **pass-through conduit** that streams the response body from NCBI directly to the client.
- Is **only used for public data** (GEO accession numbers); user-uploaded local files never touch this proxy.

3.3 The Statistical Engine

Because R cannot run natively in the browser, BioInsight 360 implements all statistical algorithms from scratch in pure JavaScript. This section describes the mathematical foundations of each implementation.

3.3.1 Normalization Methods

Counts Per Million (CPM):

$$\text{CPM}_{ij} = \frac{K_{ij}}{\sum_{i=1}^G K_{ij}} \times 10^6$$

where K_{ij} is the raw count for gene i in sample j , and G is the total number of genes. CPM normalizes for

sequencing depth but does not account for RNA composition bias.

Upper Quartile (UQ):

The 75th percentile of the non-zero counts in each sample is used as the sample-specific size factor. This is more robust to highly expressed genes that can skew the library size:

$$\hat{s}_j = Q\{0.75\}(\{K_{ij} : K_{ij} > 0\})$$

Median of Ratios (DESeq2-style):

This method computes a pseudo-reference sample using the geometric mean of each gene across all samples. The size factor for each sample is the median of the ratios of its counts to this pseudo-reference:

$$s_j = \text{median}_i \left(\frac{K_{ij}}{\tilde{K}_i} \right) \\ \text{where } \tilde{K}_i = \left(\prod_{j=1}^m K_{ij} \right)^{1/m}$$

3.3.2 Statistical Tests

Welch's t-test:

Unlike the Student's t-test, Welch's t-test does not assume equal variances between the two groups, making it more appropriate for biological data:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

The degrees of freedom are calculated using the Welch-Satterthwaite equation:

$$\nu = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}}$$

The p-value is computed as $p = 2 \cdot (1 - F_t(|t|, \nu))$, where F_t is the CDF of the Student's t-distribution, approximated using a regularized incomplete beta function implementation in `lib/mathUtils.js`.

Mann-Whitney U Test:

For data that heavily violates normality assumptions, the Mann-Whitney U test provides a non-parametric alternative based on rank sums:

$$U_1 = R_1 - \frac{n_1(n_1 + 1)}{2}$$

where R_1 is the sum of ranks for group 1. For large samples, the z-statistic is:

$$z = \frac{U - \mu_U}{\sigma_U} \quad \text{where} \quad \mu_U = \frac{n_1 n_2}{2}, \quad \sigma_U = \sqrt{\frac{n_1 n_2 (n_1 + n_2 + 1)}{12}}$$

3.3.3 Multiple Testing Correction: Benjamini-Hochberg FDR

When testing tens of thousands of genes simultaneously, the probability of obtaining false positives purely by chance is extremely high. The Benjamini-Hochberg (BH) procedure controls the False Discovery Rate (FDR):

1. Sort the m p-values in ascending order: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$.

2. For each i from m down to 1, compute:

$$p_{(i)}^{\text{adj}} = \min\left(p_{(i)} \cdot \frac{m}{i}, p_{(i+1)}^{\text{adj}}\right).$$

3. A gene is declared significant if $p^{\text{adj}} \leq \alpha$ (typically 0.05).

3.3.4 Effect Size: Cohen's d

In addition to statistical significance, BioInsight 360 computes **Cohen's d** as a measure of effect size:

$$d = \frac{\bar{X}_1 - \bar{X}_2}{s_p} \quad \text{where} \\ s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

3.4 The Multi-Method Consensus Engine

3.4.1 Rationale

Different normalization methods and statistical tests can yield different sets of DEGs. A gene declared significant by DESeq2 may not be significant under edgeR, or vice versa. Relying on a single method introduces method-specific bias. BioInsight 360's consensus engine addresses this by running **six pipelines** (3 normalizations x 2 tests) and computing the intersection.

3.4.2 Consensus Score

For each gene, the **consensus score** is defined as the number of pipelines (out of 6) in which the gene passes both the log2 fold-change threshold ($|\log_2\text{FC}| \geq \theta_{\text{FC}}$) and the

adjusted p-value threshold ($p_{\text{adj}} \leq \theta_p$). Genes are then classified:

Consensus Score (as % of pipelines)	Classification
$\geq 80\%$	High Confidence
50% – 79%	Moderate Confidence
15% – 49%	Method Sensitive
< 15%	Not Significant

Table 3.2: Gene classification based on consensus score.

3.4.3 Fleiss' Kappa

To quantify the overall level of agreement among the six pipelines, BioInsight 360 computes **Fleiss' Kappa** (κ), a standard measure of inter-rater reliability for multiple raters (pipelines) classifying subjects (genes) into categories (significant / not significant):

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

Where:

- $\bar{P}$ is the mean proportion of pairwise agreements across all genes.
- $\bar{P}_e$ is the expected proportion of agreement by chance.

A κ value close to 1 indicates near-perfect agreement (all pipelines agree), while a value near 0 indicates agreement no better than chance.

3.4.4 Pairwise Jaccard Similarity

To visualize the similarity between individual pipeline results, BioInsight 360 computes the **Jaccard Index** for each pair of pipelines:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where A and B are the sets of DEGs identified by pipelines A and B , respectively. The resulting 6×6 Jaccard matrix is displayed as a heatmap.

Chapter 4: Implementation

This chapter provides a detailed walkthrough of the actual source code of BioInsight 360. All code presented here is the genuine production code from the repository, not pseudocode or simplified illustrations.

4.1 Project Structure

The BioInsight 360 codebase is organized as follows:

```
bioinsight-360-app/  
├─ app/  
│   ├─ page.js          # Landing page  
│   └─ upload/page.js   # Upload & Dataset  
Configuration  
│   └─ qc/page.js       # Quality Control  
Dashboard  
│   └─ consensus/page.js # Consensus DGE  
Dashboard  
│   └─ pathways/page.js  # Pathway Enrichment  
│   └─ export/page.js    # PDF Report  
Generation  
│   └─ api/ncbi/         # CORS proxy API  
route  
├─ lib/  
│   └─ storage.js        # Dual-layer state  
management  
│   └─ normalize.js      # CPM, UQ, Median of  
Ratios  
│   └─ statistics.js     # Welch's t-test,
```

```

Mann-Whitney U, BH FDR
|   └─ consensus.js      # Multi-method
consensus engine
|   └─ mathUtils.js      # Low-level math
(mean, variance, tDistCDF, rank)
|   └─ pca.js            # Principal
Component Analysis
|   └─ qc.js              # Library size,
detection rate, correlation
|   └─ cpm_filter.js      # Low-expression
gene filtering
|   └─ geo_api.js         # NCBI GEO metadata
fetcher
|   └─ enrichment.js      # GO enrichment
logic
|   └─ quickgo_api.js     # QuickGO REST API
client
└─ components/           # React
visualization components
|   └─ ConsensusVolcano.js
|   └─ AgreementHeatmap.js
|   └─ UpSetPlot.js
|   └─ JaccardMatrix.js
|   └─ DEGTable.js
|   └─ PCAScatter.js
└─ package.json

```

4.2 The Storage Module

(**lib/storage.js**)

The storage module is the foundation of the zero-server architecture:

```
const memoryStore = new Map();

export const Storage = {
  setItem: (key, value) => {
    // Always store in memory for
    unlimited size (works across Next.js Link
    navigations)
    memoryStore.set(key, value);

    // Try to persist to sessionStorage
    to survive F5 reloads, but catch quota
    errors silently
    try {
      sessionStorage.setItem(key,
value);
    } catch (e) {
      console.warn(`Storage quota
exceeded for ${key}. Data stored in memory
only. A page refresh will clear this
data.`);
    }
  },
  getItem: (key) => {
    // Memory takes precedence
    if (memoryStore.has(key)) {
      return memoryStore.get(key);
    }

    // Fallback to sessionStorage
    if (typeof window !== 'undefined') {
      const val =
sessionStorage.getItem(key);
      if (val !== null) {
        memoryStore.set(key, val);
      }
    }
  }
};
```

```

        }
        return val;
    }
    return null;
},
removeItem: (key) => {
    memoryStore.delete(key);
    if (typeof window !== 'undefined') {
        sessionStorage.removeItem(key);
    }
},
clear: () => {
    memoryStore.clear();
    if (typeof window !== 'undefined') {
        sessionStorage.clear();
    }
}
};

```

Code Listing 4.1: The complete `lib/storage.js` module. Note the dual-layer approach: `memoryStore` (a `Map`) is the primary data store, with `sessionStorage` as a persistence fallback. The `try/catch` around `sessionStorage.setItem` silently handles quota exceeded errors for large datasets.

4.3 The Normalization Module

(`lib/normalize.js`)

This module implements all three normalization methods:

```

import { sum, quantile, geometricMean,
median } from './mathUtils.js';

/**
 * Normalizes a gene expression matrix using
Counts Per Million (CPM).
 * @param {Array<Array<number>>} matrix -
Genes (rows) x Samples (cols)
 * @returns {Array<Array<number>>}}
Normalized matrix
 */
export function normalizeCPM(matrix) {
  if (!matrix || matrix.length === 0 ||
matrix[0].length === 0) return [];
  const numGenes = matrix.length;
  const numSamples = matrix[0].length;
  const colSums = new
Array(numSamples).fill(0);

  for (let j = 0; j < numSamples; j++) {
    for (let i = 0; i < numGenes; i++) {
      colSums[j] += matrix[i][j];
    }
  }

  const normMatrix = Array.from({ length:
numGenes }, () => new Array(numSamples));
  for (let i = 0; i < numGenes; i++) {
    for (let j = 0; j < numSamples; j++) {
      normMatrix[i][j] = colSums[j] > 0 ?
(matrix[i][j] / colSums[j]) * 1e6 : 0;
    }
  }
  return normMatrix;
}

```

```

}

/**
 * Normalizes a gene expression matrix using
Upper Quartile (UQ) method.
 * @param {Array<Array<number>>} matrix -
Genes x Samples
 * @returns {Array<Array<number>>}
Normalized matrix
 */
export function
normalizeUpperQuartile(matrix) {
  if (!matrix || matrix.length === 0 ||
matrix[0].length === 0) return [];
  const numGenes = matrix.length;
  const numSamples = matrix[0].length;
  const sizeFactors = new
Array(numSamples).fill(1);

  for (let j = 0; j < numSamples; j++) {
    const nonZeros = [];
    for (let i = 0; i < numGenes; i++) {
      if (matrix[i][j] > 0)
nonZeros.push(matrix[i][j]);
    }
    const uq = quantile(nonZeros, 0.75);
    sizeFactors[j] = uq > 0 ? uq : 1;
  }

  const meanUQ = sum(sizeFactors) /
sizeFactors.length;
  const normMatrix = Array.from({ length:
numGenes }, () => new Array(numSamples));
  for (let i = 0; i < numGenes; i++) {

```

```

        for (let j = 0; j < numSamples; j++) {
            normMatrix[i][j] = matrix[i][j] /
(sizeFactors[j] / meanUQ);
        }
    }
    return normMatrix;
}

/**
 * Normalizes a gene expression matrix using
Median of Ratios (DESeq2 style).
 * @param {Array<Array<number>>} matrix -
Genes x Samples
 * @returns {Array<Array<number>>}
Normalized matrix
 */
export function
normalizeMedianOfRatios(matrix) {
    if (!matrix || matrix.length === 0 ||
matrix[0].length === 0) return [];
    const numGenes = matrix.length;
    const numSamples = matrix[0].length;
    const geoMeans = new Array(numGenes);

    for (let i = 0; i < numGenes; i++) {
        geoMeans[i] = geometricMean(matrix[i]);
    }

    const sizeFactors = new
Array(numSamples).fill(1);
    for (let j = 0; j < numSamples; j++) {
        const ratios = [];
        for (let i = 0; i < numGenes; i++) {
            if (geoMeans[i] > 0 && matrix[i][j] >=

```

```

0) {
    ratios.push(matrix[i][j] /
geoMeans[i]);
    }
    }
    const sf = median(ratios);
    sizeFactors[j] = sf > 0 ? sf : 1;
}

const normMatrix = Array.from({ length:
numGenes }, () => new Array(numSamples));
for (let i = 0; i < numGenes; i++) {
    for (let j = 0; j < numSamples; j++) {
        normMatrix[i][j] = matrix[i][j] /
sizeFactors[j];
    }
}
return normMatrix;
}

```

Code Listing 4.2: The complete `lib/normalize.js` module implementing CPM, Upper Quartile, and Median of Ratios normalization.

4.4 The Statistical Testing Module (`lib/statistics.js`)

This module contains the core statistical implementations:

```

import { mean, variance, tDistCDF, rank,
normalCDF } from './mathUtils.js';

```

```

/**
 * Perform Welch's t-test for two unequal
variance samples.
 */
export function welchTTest(group1, group2) {
  const m1 = mean(group1);
  const m2 = mean(group2);
  const v1 = variance(group1);
  const v2 = variance(group2);
  const n1 = group1.length;
  const n2 = group2.length;

  if (n1 < 2 || n2 < 2 || (v1 === 0 && v2
=== 0)) return { t: 0, df: 1, pvalue: 1 };

  const t = (m1 - m2) / Math.sqrt((v1 / n1)
+ (v2 / n2));
  const dfNum = Math.pow((v1 / n1) + (v2 /
n2), 2);
  const dfDen = Math.pow(v1 / n1, 2) / (n1 -
1) + Math.pow(v2 / n2, 2) / (n2 - 1);
  const df = dfNum / dfDen;

  const pvalue = 2 * (1 -
tDistCDF(Math.abs(t), df));
  return { t, df, pvalue };
}

/**
 * Perform Mann-Whitney U test (Normal
approximation).
 */
export function mannWhitneyU(group1, group2)
{

```

```

    const n1 = group1.length;
    const n2 = group2.length;
    if (n1 === 0 || n2 === 0) return { U: 0,
z: 0, pvalue: 1 };

    const all = [...group1, ...group2];
    const ranks = rank(all);
    let R1 = 0;
    for (let i = 0; i < n1; i++) R1 +=
ranks[i];

    const U1 = R1 - (n1 * (n1 + 1)) / 2;
    const U2 = n1 * n2 - U1;
    const U = Math.min(U1, U2);

    const mU = (n1 * n2) / 2;
    const sigmaU = Math.sqrt((n1 * n2 * (n1 +
n2 + 1)) / 12);
    const z = sigmaU === 0 ? 0 : (U - mU) /
sigmaU;

    const pvalue = 2 * normalCDF(-
Math.abs(z));
    return { U, z, pvalue };
}

/**
 * Calculate log2 Fold Change with
pseudocounts.
 */
export function log2FoldChange(treated,
control) {
    const m1 = mean(treated);
    const m2 = mean(control);

```

```

    return Math.log2((m1 + 1) / (m2 + 1));
}

/**
 * Calculate Cohen's d effect size.
 */
export function cohensD(group1, group2) {
    const n1 = group1.length;
    const n2 = group2.length;
    if (n1 < 2 || n2 < 2) return 0;

    const v1 = variance(group1);
    const v2 = variance(group2);
    const pooledSd = Math.sqrt(((n1 - 1) * v1
+ (n2 - 1) * v2) / (n1 + n2 - 2));
    if (pooledSd === 0) return 0;
    return (mean(group1) - mean(group2)) /
pooledSd;
}

/**
 * Apply Benjamini-Hochberg FDR correction.
 */
export function benjaminiHochberg(pvalues) {
    const n = pvalues.length;
    const sorted = pvalues.map((p, i) => ({ p,
i })).sort((a, b) => a.p - b.p);
    const padj = new Array(n);
    let minPrev = 1;

    for (let i = n - 1; i >= 0; i--) {
        const q = (sorted[i].p * n) / (i + 1);
        minPrev = Math.min(minPrev, q);
        padj[sorted[i].i] = minPrev;
    }
}

```

```

    }
    return padj;
}

/**
 * Run differential expression for all
genes.
 */
export function
runDifferentialExpression(matrix,
groupIndices1, groupIndices2, test =
'welch') {
    if (!matrix || matrix.length === 0) return
[];
    const results = [];
    const pvalues = [];

    for (let i = 0; i < matrix.length; i++) {
        const geneExpr = matrix[i];
        const g1 = groupIndices1.map(idx =>
geneExpr[idx]);
        const g2 = groupIndices2.map(idx =>
geneExpr[idx]);

        const l2fc = log2FoldChange(g1, g2);
        const d = cohensD(g1, g2);
        let pval = 1;

        if (test === 'welch') {
            pval = welchTTest(g1, g2).pvalue;
        } else {
            pval = mannWhitneyU(g1, g2).pvalue;
        }
    }
}

```

```

    pvalues.push(pval);
    results.push({ gene_index: i, log2fc:
l2fc, pvalue: pval, padj: 1, cohens_d: d });
  }

  const padjs = benjaminiHochberg(pvalues);
  for (let i = 0; i < results.length; i++) {
    results[i].padj = padjs[i];
  }

  return results;
}

```

Code Listing 4.3: The complete `lib/statistics.js` module. Note the implementation of Welch's t-test with Welch-Satterthwaite degrees of freedom, the Mann-Whitney U test with normal approximation, Cohen's d effect size, and Benjamini-Hochberg FDR correction.

4.5 The Consensus Engine

(`lib/consensus.js`)

The consensus engine ties everything together:

```

import { normalizeCPM,
normalizeUpperQuartile,
normalizeMedianOfRatios } from
'./normalize.js';
import { runDifferentialExpression } from
'./statistics.js';
import { median } from './mathUtils.js';

```

```

/**
 * Run all normalization and testing
pipelines.
 */
export function runAllPipelines(rawMatrix,
geneNames, groupIndices1, groupIndices2) {
  const normalizers = [
    { name: 'CPM', fn: normalizeCPM },
    { name: 'UQ', fn: normalizeUpperQuartile
},
    { name: 'MedianRatios', fn:
normalizeMedianOfRatios }
  ];
  const tests = ['welch', 'mannWhitney'];

  const pipelines = [];
  for (const norm of normalizers) {
    const normMatrix = norm.fn(rawMatrix);
    for (const test of tests) {
      const deRes =
runDifferentialExpression(normMatrix,
groupIndices1, groupIndices2, test);
      pipelines.push({ name:
`${norm.name}_${test}`, results: deRes });
    }
  }
  return { pipelines, geneNames };
}

/**
 * Format pre-computed downstream DEGs
(DESeq2, edgeR, limma) into pipeline
structure.
 */

```

```

export function
formatDownstreamPipelines(deseq2, edger,
limma) {
  const geneSet = new Set();
  const extract = (data) => {
    if(!data) return;
    data.forEach(row => {
      const g = row.Gene_ID || row.Gene ||
row.id || row.ID;
      if (g) geneSet.add(g);
    });
  };

  extract(deseq2);
  extract(edger);
  extract(limma);

  const geneNames = Array.from(geneSet);
  const geneToIndex = new Map();
  geneNames.forEach((g, idx) =>
geneToIndex.set(g, idx));

  const createResults = (data) => {
    const results = new
Array(geneNames.length).fill(null);
    if (!data) return results;

    data.forEach(row => {
      const g = row.Gene_ID || row.Gene ||
row.id || row.ID;
      const idx = geneToIndex.get(g);
      if (idx !== undefined) {
        results[idx] = {
          gene_index: idx,

```

```

        log2fc: parseFloat(row.logFC ||
row.log2FoldChange || 0),
        padj: parseFloat(row.padj ||
row.FDR || row.pvalue || 1)
    };
    }
    });
    return results;
};

const pipelines = [];
if (deseq2) pipelines.push({ name:
'DESeq2', results: createResults(deseq2) });
if (edgeR) pipelines.push({ name: 'edgeR',
results: createResults(edgeR) });
if (limma) pipelines.push({ name: 'limma',
results: createResults(limma) });

return { pipelines, geneNames };
}

/**
 * Classify a gene based on its consensus
score, dynamically scaled to total
pipelines.
 */
export function classifyGene(consensusScore,
totalPipelines = 6) {
    const ratio = consensusScore /
totalPipelines;
    if (ratio >= 0.8) return
'high_confidence'; // 80%+ agreement
    if (ratio >= 0.5) return
'moderate_confidence'; // 50%+ agreement

```

```

    if (ratio >= 0.15) return
'method_sensitive';          // At least 1
pipeline
    return 'not_significant';
}

/**
 * Compute multi-pipeline consensus for each
gene.
 */
export function computeConsensus(pipelines,
geneNames, fcThreshold, pThreshold) {
    if (!pipelines || pipelines.length === 0
|| !geneNames) return [];
    const numGenes = geneNames.length;
    const consensusResults = [];

    for (let i = 0; i < numGenes; i++) {
        let sigCount = 0;
        const l2fcs = [];
        const pvals = [];
        let posDirs = 0;
        let negDirs = 0;

        for (const pipe of pipelines) {
            const res = pipe.results[i];
            if (res) {
                l2fcs.push(res.log2fc);
                pvals.push(res.padj);

                if (res.padj <= pThreshold &&
Math.abs(res.log2fc) >= fcThreshold) {
                    sigCount++;
                    if (res.log2fc > 0) posDirs++;

```

```

        else if (res.log2fc < 0)
negDirs++;
    }
}
}

consensusResults.push({
    geneName: geneNames[i],
    consensusScore: sigCount,
    category: classifyGene(sigCount,
pipelines.length),
    log2fc_median: median(l2fcs),
    pvalues: pvals,
    directions: { positive: posDirs,
negative: negDirs }
    });
}

return consensusResults;
}

/**
 * Compute Jaccard index between two sets.
 */
export function computeJaccard(setA, setB) {
    if (!setA || !setB) return 0;
    const intersection = new
Set([...setA].filter(x => setB.has(x)));
    const union = new Set([...setA, ...setB]);
    if (union.size === 0) return 1;
    return intersection.size / union.size;
}

/**

```

```

    * Compute overlap coefficient between two
    sets.
    */
export function
computeOverlapCoefficient(setA, setB) {
    if (!setA || !setB) return 0;
    const intersection = new
Set([...setA].filter(x => setB.has(x)));
    const minSize = Math.min(setA.size,
setB.size);
    if (minSize === 0) return 1;
    return intersection.size / minSize;
}

/**
    * Compute Fleiss' Kappa for inter-rater
    agreement.
    * @param {Array<Array<number>>}<number> matrix -
    Genes x Pipelines (0 or 1 values)
    */
export function computeFleissKappa(matrix) {
    if (!matrix || matrix.length === 0) return
0;
    const N = matrix.length;
    const n = matrix[0].length;
    if (n === 0) return 0;

    const categoryCounts = [0, 0];
    const P = new Array(N).fill(0);

    for (let i = 0; i < N; i++) {
        let ones = 0;
        for (let j = 0; j < n; j++) {
            if (matrix[i][j]) {

```

```

        ones++;
        categoryCounts[1]++;
    } else {
        categoryCounts[0]++;
    }
}

P[i] = (ones * ones + (n - ones) * (n -
ones) - n) / (n * (n - 1));
}

const P_bar = P.reduce((a, b) => a + b, 0)
/ N;

const Pe = Math.pow(categoryCounts[0] / (N
* n), 2) + Math.pow(categoryCounts[1] / (N *
n), 2);

if (Pe === 1) return 1;
return (P_bar - Pe) / (1 - Pe);
}

/**
 * Compute pairwise Jaccard similarity
between all pipelines.
 */
export function
computePairwiseJaccard(pipelines,
fcThreshold, pThreshold) {
    if (!pipelines || pipelines.length === 0)
return [];
    const sigSets = pipelines.map(pipe => {
        const s = new Set();
        pipe.results.forEach(res => {
            if (res && res.padj <= pThreshold &&
Math.abs(res.log2fc) >= fcThreshold)

```

```

s.add(res.gene_index);
});
return s;
});

const n = sigSets.length;
const matrix = Array.from({ length: n },
() => new Array(n).fill(1));

for (let i = 0; i < n; i++) {
  for (let j = i + 1; j < n; j++) {
    const jaccard =
computeJaccard(sigSets[i], sigSets[j]);
    matrix[i][j] = jaccard;
    matrix[j][i] = jaccard;
  }
}
return matrix;
}

```

Code Listing 4.4: The complete `lib/consensus.js` module. The `runAllPipelines` function iterates over 3 normalizations $\times$ 2 tests = 6 pipelines. The `computeConsensus` function tallies how many pipelines classify each gene as significant. The `computeFleissKappa` function calculates inter-rater reliability.

Chapter 5: Software Testing & Validation

5.1 Testing Strategy

The testing strategy for BioInsight 360 encompasses three dimensions:

1. **Functional Testing:** Verifying that each module (upload, parsing, normalization, statistical testing, consensus) produces correct outputs.
2. **Performance Benchmarking:** Measuring memory consumption and execution speed across varying dataset sizes.
3. **Edge-Case Testing:** Evaluating the system's behavior with malformed inputs, extreme values, and boundary conditions.

5.2 Functional Testing

5.2.1 File Format Compatibility

The upload module was tested with the following file formats:

Input Format	Delimiter	Compressed?	Result
Standard CSV	Comma	No	Parsed correctly

TSV (tab-delimited)	Tab	No	Parsed correctly (auto-detected)
R <code>write.table</code> output	Space/Tab	No	Parsed correctly (header-fix applied)
GEO Series Matrix	Tab	.gz	Decompressed and parsed correctly
Malformed CSV (empty rows)	Comma	No	Handled gracefully (skipEmptyLines)

Table 5.1: File format compatibility testing results.

5.2.2 R `write.table` Header Fix

A common problem with RNA-Seq data distributed via GEO is that the count matrices are exported from R using `write.table()`, which produces a file where the header row has one fewer column than the data rows (because R treats row names as a special unnamed column). BioInsight 360's `fixMissingHeader()` function detects this mismatch and prepends a "Gene" header, ensuring correct parsing.

5.3 Performance Benchmarking

Note: The benchmarks presented in this section represent the expected performance characteristics based on the algorithmic complexity of the implementation. Formal benchmarks with

`performance.now()` instrumentation are planned for the final testing phase.

5.3.1 Algorithmic Complexity Analysis

Operation	Time Complexity	Space Complexity
CSV Parsing (PapaParse)	$O(n)$	$O(n)$
CPM Normalization	$O(G \times S)$	$O(G \times S)$
Upper Quartile Normalization	$O(G \times S + S \times G \log G)$	$O(G \times S)$
Median of Ratios	$O(G \times S)$	$O(G \times S)$
Welch's t-test (per gene)	$O(S)$	$O(1)$
Full pipeline (6 pipelines)	$O(6 \times G \times S)$	$O(G \times S)$
Consensus computation	$O(G \times P)$	$O(G)$
Fleiss' Kappa	$O(G \times P)$	$O(G)$
Jaccard Matrix	$O(P^2 \times G)$	$O(P^2)$

Table 5.2: Algorithmic complexity analysis. G = number of genes, S = number of samples, P = number of pipelines.

Where $G = 15,000$ genes and $S = 12$ samples, the total work for the full consensus pipeline is approximately $6 \times 15,000 \times 12 = 1,080,000$ operations — well within the capability of a modern browser executing millions of operations per millisecond.

5.4 Edge-Case Testing

Test Case	Expected Behavior	Actual Behavior
Zero-variance gene (all counts identical)	t-statistic = 0, p = 1	Correctly handled (line 14 of statistics.js)
All-zero gene	log2FC = 0 (pseudocount prevents log(0))	Correctly handled
Single sample per group (n=1)	Cannot compute variance; returns p = 1	Correctly handled (line 14 of statistics.js)
sessionStorage quota exceeded	Data persists in memory only; warning logged	Correctly handled (line 12 of storage.js)
File with only 1 column detected	Falls back to space/tab normalization	Correctly handled (line 203-208 of upload/page.js)

Table 5.3: Edge-case testing results.

Chapter 6: Results & Discussion

6.1 Case Study

Note: This section will present the results of running BioInsight 360 on the GSE52778 dataset (a breast cancer RNA-Seq dataset) during the formal testing phase. The following subsections outline the analysis plan and expected deliverables.

6.1.1 Dataset Description

The GSE52778 dataset, available from the NCBI Gene Expression Omnibus, contains RNA-Seq count data from breast cancer cell lines. It includes biological replicates for both control and treated conditions, making it ideal for evaluating DGE tools.

6.1.2 Analysis Plan

1. Load the GSE52778 count matrix into BioInsight 360.
2. Perform Quality Control: Generate library size distributions and PCA plots.
3. Run the Consensus Engine with default thresholds ($|\log_2\text{FC}| \geq 1.0$, $p_{\text{adj}} \leq 0.05$).
4. Capture the Volcano plot, Jaccard heatmap, and UpSet plot.
5. Export the top DEGs table and compare with published literature for this dataset.

6.1.3 Expected Deliverables

- Screenshot of the Volcano Plot showing the distribution of DEGs.
- Jaccard Similarity Heatmap showing inter-pipeline agreement.
- Table of the top 50 high-confidence consensus DEGs with gene names, median log2FC, and consensus scores.
- Comparison of our results with the original publication's DEG list.

6.2 Limitations

Despite the architectural innovations, BioInsight 360 has several limitations that must be acknowledged:

1. **No Raw Read Alignment:** The platform requires pre-processed count matrices as input. It cannot align raw FASTQ reads, as genome alignment algorithms (e.g., STAR, HISAT2) require gigabytes of reference genome indices and hours of computation — currently infeasible in a browser.
2. **Simplified Statistical Model:** The in-browser implementation uses Welch's t-test and Mann-Whitney U, which are general-purpose statistical tests. These do not account for the specific distributional properties of RNA-Seq data (e.g., the negative binomial distribution used by DESeq2 and edgeR). This means the p-values

may differ from those produced by DESeq2/edgeR, particularly for genes with very low counts.

3. Browser Memory Constraints: While the in-memory Map store can theoretically hold large datasets, browsers impose practical limits on JavaScript heap size (typically 1–4 GB). For very large single-cell RNA-Seq datasets (e.g., 50,000 cells x 30,000 genes), the browser may run out of memory.

4. Single-Threaded Execution: The current implementation runs all computations on the browser's main thread. For large datasets, this can cause the UI to become unresponsive during computation. Future iterations should migrate heavy computation to Web Workers.

Chapter 7: Conclusion & Future Scope

7.1 Summary of Contributions

This dissertation has presented BioInsight 360, a novel bioinformatics platform that makes three key contributions:

1. **Privacy-First Architecture:** By executing all computation in the browser, BioInsight 360 is the first DGE analysis platform to guarantee zero-server data privacy by design. This makes it inherently compliant with HIPAA and GDPR, without requiring any special infrastructure or legal agreements.
2. **Multi-Method Consensus Engine:** The six-pipeline consensus approach — quantified by Fleiss' Kappa and Jaccard Similarity — provides a principled way to distinguish high-confidence DEGs from method-sensitive false positives. This addresses a well-known weakness of single-method analysis.
3. **Accessible Design:** By providing a modern, interactive web interface with Volcano plots, Heatmaps, and UpSet plots, BioInsight 360 makes DGE analysis accessible to researchers without programming expertise.

7.2 Future Work

Several directions for future development have been identified:

1. **Web Worker Integration:** Migrating the statistical engine to a dedicated Web Worker thread will prevent UI blocking during computation, enabling a smooth 60 FPS user experience even during heavy analysis.
2. **WebAssembly (Wasm) Compilation:** Compiling established C/C++ bioinformatics tools (e.g., `kallisto` for pseudoalignment, or a lightweight NB-GLM solver) to WebAssembly would enable near-native speed computation and more statistically rigorous DGE analysis directly in the browser.
3. **IndexedDB for Large-Scale Storage:** Migrating from the current `Map / sessionStorage` approach to IndexedDB would enable persistent storage of datasets up to several gigabytes, supporting single-cell RNA-Seq analysis.
4. **Offline Support via Service Workers:** Registering a Service Worker would enable BioInsight 360 to function as a Progressive Web App (PWA), allowing users to perform analysis even without an internet connection.
5. **Collaborative Analysis:** Integrating WebRTC for peer-to-peer data sharing would allow multiple researchers to collaboratively analyze the same dataset without any data touching a central server.

References

- [1] Love, M. I., Huber, W., & Anders, S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12), 550. DOI: 10.1186/s13059-014-0550-8
- [2] Robinson, M. D., McCarthy, D. J., & Smyth, G. K. (2010). edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics*, 26(1), 139–140. DOI: 10.1093/bioinformatics/btp616
- [3] Ritchie, M. E., Phipson, B., Wu, D., Hu, Y., Law, C. W., Shi, W., & Smyth, G. K. (2015). limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic Acids Research*, 43(7), e47. DOI: 10.1093/nar/gkv007
- [4] Afgan, E., Baker, D., Batut, B., van den Beek, M., Bouvier, D., Cech, M., ... & Blankenberg, D. (2018). The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. *Nucleic Acids Research*, 46(W1), W537–W544. DOI: 10.1093/nar/gky379
- [5] Ge, S. X., Son, E. W., & Yao, R. (2018). iDEP: an integrated web application for differential expression and pathway analysis of RNA-Seq data. *BMC Bioinformatics*, 19(1), 534.
- [6] Xia, J., Gill, E. E., & Hancock, R. E. W. (2015). NetworkAnalyst for statistical, visual and network-based meta-analysis of gene expression data. *Nature Protocols*, 10(6), 823–844. DOI: 10.1038/nprot.2015.052

- [7] Holt, M. (n.d.). *Papa Parse: Fast and powerful CSV parser for JavaScript*. GitHub. <https://github.com/mholt/PapaParse>
- [8] Wang, Z., Gerstein, M., & Snyder, M. (2009). RNA-Seq: a revolutionary tool for transcriptomics. *Nature Reviews Genetics*, 10(1), 57–63.
- [9] Mortazavi, A., Williams, B. A., McCue, K., Schaeffer, L., & Wold, B. (2008). Mapping and quantifying mammalian transcriptomes by RNA-Seq. *Nature Methods*, 5(7), 621–628.
- [10] Dobin, A., Davis, C. A., Schlesinger, F., Drber, J., Zaleski, C., Jha, S., ... & Gingeras, T. R. (2013). STAR: ultrafast universal RNA-seq aligner. *Bioinformatics*, 29(1), 15–21.
- [11] Kim, D., Paggi, J. M., Park, C., Bennett, C., & Salzberg, S. L. (2019). Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Nature Biotechnology*, 37(8), 907–915.
- [12] Liao, Y., Smyth, G. K., & Shi, W. (2014). featureCounts: an efficient general purpose program for assigning sequence reads to genomic features. *Bioinformatics*, 30(7), 923–930.
- [13] Patro, R., Duggal, G., Love, M. I., Irizarry, R. A., & Kingsford, C. (2017). Salmon provides fast and bias-aware quantification of transcript expression. *Nature Methods*, 14(4), 417–419.
- [14] Costa-Silva, J., Domingues, D., & Lopes, F. M. (2017). RNA-Seq differential expression analysis: An extended review and a software tool. *PLOS ONE*, 12(12), e0190152.
- [15] Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple

testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289–300.

[16] Welch, B. L. (1947). The generalization of Student's problem when several different population variances are involved. *Biometrika*, 34(1–2), 28–35.

[17] Mann, H. B., & Whitney, D. R. (1947). On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, 18(1), 50–60.

[18] Fleiss, J. L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5), 378–382.

[19] Jaccard, P. (1912). The distribution of the flora in the alpine zone. *New Phytologist*, 11(2), 37–50.

[20] Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2nd ed.). Lawrence Erlbaum Associates.

[21] National Human Genome Research Institute. (2022). The Cost of Sequencing a Human Genome. <https://www.genome.gov/about-genomics/fact-sheets/Sequencing-Human-Genome-cost>

[22] Next.js Documentation. (n.d.). Vercel. <https://nextjs.org/docs>

[23] Plotly.js Documentation. (n.d.). Plotly. <https://plotly.com/javascript/>

[24] MDN Web Docs. (n.d.). DecompressionStream API. Mozilla. <https://developer.mozilla.org/en-US/docs/Web/API/DecompressionStream>

[25] MDN Web Docs. (n.d.). Web Workers API. Mozilla.
https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API